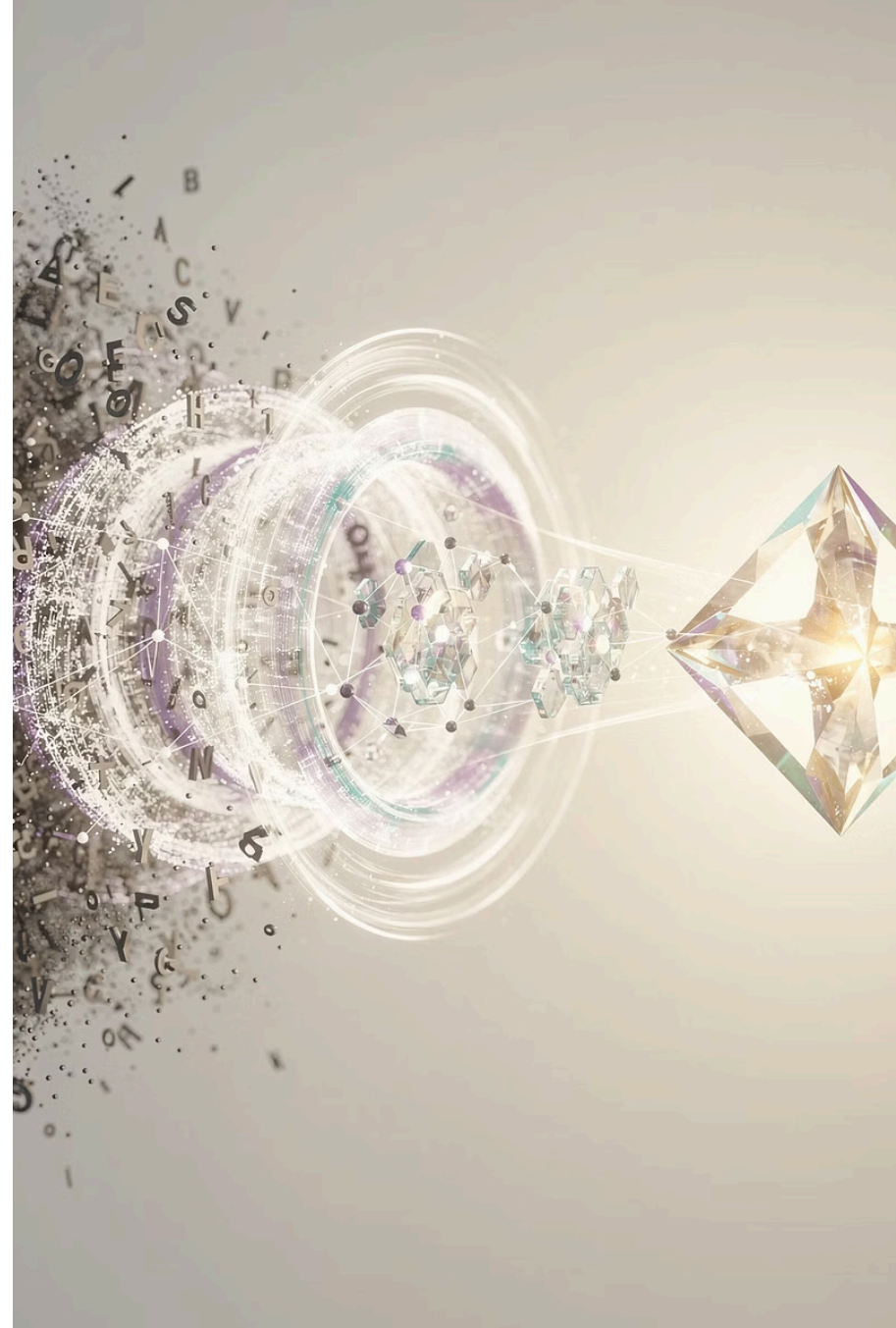


# Come nascono gli LLM

## Training Pipeline

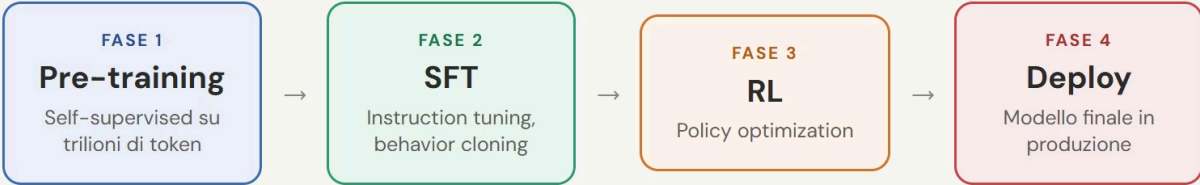


by Daniele Mantovani



# Pipeline di Training degli LLM

Dal Next Token Prediction ai Reasoning Models — un processo a stadi successivi di raffinamento



LA FASE RL SI BIFORCA IN DUE APPROCCI COMPLEMENTARI

DUE PERCORSI, UN OBIETTIVO

## ALLINEAMENTO

### RLHF

Human Feedback: annotatori umani ordinano risposte per qualità. Reward model soggettivo.

Ricompensa: gradimento umano

## RAGIONAMENTO

### RLVR

Verifiable Rewards: matematica, codice, logica. Verificatore automatico oggettivo.

Ricompensa: verità oggettiva\*

\* Solo dove esiste un verificatore automatico affidabile (math, codice, logica formale)

## CLASSICO

### Modelli Chat / General Purpose

Pre-training → SFT → RLHF → Deploy

Il modello impara a conversare (SFT), poi viene allineato alle preferenze umane (RLHF). Focus su stile, utilità e sicurezza.

## REASONING

### Reasoning Models (es. DeepSeek-R1)

Pre-training → SFT → RLVR → SFT → RLHF →

Deploy

Prima la logica (RLVR), poi il consolidamento (SFT), infine l'allineamento (RLHF). La ragione prima della forma.

#### Messaggio chiave:

Addestrare un LLM non è un singolo passaggio ma una pipeline di raffinamento progressivo. Nei reasoning models moderni, RLVR e RLHF sono applicati in sequenza complementare, non in alternativa.

# Composizione del Training Set

Non solo "tutto internet" — un mix bilanciato di fonti per varietà e qualità

MIX TIPICO DEI DATI DI PRE-TRAINING



|                                                                                     |                                                                                                |         |
|-------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------|---------|
|    | <b>Web filtrato</b><br>Common Crawl, FineWeb — miliardi di pagine deduplicate                  | ~50-60% |
|    | <b>Libri e articoli scientifici</b><br>Linguaggio curato, alta qualità, conoscenza strutturata | ~15-20% |
|    | <b>Codice sorgente (GitHub)</b><br>Decine di linguaggi — migliora ragionamento logico          | ~10-15% |
|  | <b>Conversazioni e Q&amp;A</b><br>Reddit, StackOverflow — struttura dialogica                  | ~5-10%  |

ORDINE DI GRANDEZZA

**1 – 15T token**

Chinchilla (2022): ~1.4T

Llama 3 (2024): 15T

≈ 15 milioni di libri da 200 pagine

TENDENZA 2024-26

**Qualità > Quantità**

I dati web "facili" sono quasi esauriti. Focus su dati sintetici, curated datasets e partnership con editori.

**Il mix è tutto:** più codice → migliore ragionamento. Più multilingua → migliore traduzione. La composizione del training set è un iperparametro critico.

# Pipeline di Curatela: dalla Materia Prima al Dataset Pulito

Pulizia, filtraggio qualità, sicurezza — e i rischi di non farlo

## 1. LA MATERIA PRIMA

### Web filtrato

C4, FineWeb, CommonCrawl

### Testi di alta qualità

Libri, articoli scientifici, enciclopedie

### Codice sorgente

GitHub, Stack Overflow

### Dati multilingua

Wikipedia, testi in 100+ lingue

*Mix bilanciato di fonti per garantire varietà e qualità*

## 2. PIPELINE DI CURATELA E SICUREZZA

### 1 Pulizia e deduplicazione

Rimozione boilerplate, HTML, duplicati esatti e near-duplicate (MinHash + LSH). Previene memorizzazione e overfitting.

### 2 Filtraggio della qualità

Classificatori assegnano un quality score. Si scartano spam, testi incoerenti e contenuti auto-generati di bassa qualità.

### 3 Rimozione contenuti dannosi + PII

Filtro per tossicità, CSAM, odio. Mascheramento di dati personali (email, telefoni, indirizzi).

### Rischio: Model Collapse

Senza filtraggio qualità, il modello converge su pattern ripetitivi e perde diversità. Come fotocopiare una fotocopia: ogni iterazione perde dettaglio.

## 3. FASI DI ADDESTRAMENTO

### Dataset curato e bilanciato

Triloni di token puliti, diversificati e sicuri — pronti per il pre-training

- **Pre-training** Triloni di token
- **Mid-training** Dati di alta qualità
- **Fine-tuning** Dataset specifico e piccolo

### Knowledge Cutoff

La conoscenza del modello si ferma alla data di raccolta dei dati. Tutto ciò che accade dopo è invisibile.

### Fonti delle allucinazioni:

- Knowledge cutoff — fatti post-training inesistenti
- Objective mismatch — il modello ottimizza plausibilità, non verità
- Bias del reward model (RLHF) — premia la forma, non la correttezza
- Sampling stocastico — genera token probabili, non necessariamente giusti

Soluzioni: RAG, web search, tool use.

# L'Obiettivo del Pre-training: Self-Supervised Learning e Cross-Entropy Loss

Come un compito apparentemente semplice produce capacità sorprendenti

## PERCHÉ È SELF-SUPERVISED

Il testo **genera automaticamente le proprie label**. Non servono annotatori umani: ogni frase è sia input che target, spostato di una posizione.

Esempio — dalla frase: "Il gatto è sul tetto"

Input: Il gatto è sul

Target: gatto è sul tetto

- 1 Il modello vede "Il" e deve prevedere "gatto"
  - 2 Vede "Il gatto" e deve prevedere "è"
  - 3 Vede "Il gatto è sul" e deve prevedere "tetto"
- ✓ **Nessun umano ha etichettato nulla** — la supervisione viene dal testo stesso

## LA MISURA DELL'ERRORE: CROSS-ENTROPY LOSS

$$\mathcal{L} = -\log p_{\theta}(x_{\text{corretto}})$$

Per ogni token, il modello produce una **distribuzione di probabilità** su tutto il vocabolario. La loss misura quanto la probabilità assegnata al **token corretto** è lontana da 1. Poiché la ground truth è one-hot, la formula si semplifica al **logaritmo negativo** della probabilità del token giusto.

### ● Modello sicuro

$p(\text{tetto}) = 0.95$   
 $\mathcal{L} = -\log(0.95) \approx 0.05$   
→ Loss quasi zero

### ● Modello incerto

$p(\text{tetto}) = 0.01$   
 $\mathcal{L} = -\log(0.01) \approx 4.6$   
→ Loss molto alta

## IL RISULTATO EMERGENTE

Ottimizzando questo obiettivo apparentemente semplice, il modello impara progressivamente:



Sintassi e grammatica



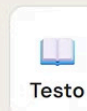
Relazioni semantiche



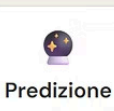
Conoscenza fattuale



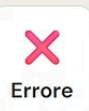
Ragionamento logico



Testo



Predizione



Errore



Correzione pesi



Ripetuto su trilioni di token per settimane/mesi

# Pre-training e Continued Pre-training

Compressione della conoscenza e rappresentazione latente — l'iceberg sotto la superficie

## SOPRA LA SUPERFICIE — CIÒ CHE VEDIAMO

### Testo Generato (Comportamento)

Le risposte che il modello produce. Ciò che è visibile all'utente.  
Modellato dal post-training (SFT, RLHF).

## SOTTO LA SUPERFICIE — LA VERA CONOSCENZA

### Conoscenza Latente Compressa

Codificata nei pesi del modello durante il pre-training. Struttura del linguaggio, semantica, fatti, relazioni — tutto compresso in miliardi di parametri.

### Rappresentazioni interne

Embedding, pattern di attivazione, circuiti interni. Il modello non memorizza esplicitamente i testi: ne estrae e comprime la struttura statistica profonda.

**Attenzione — memorization:** in rari casi il modello può riprodurre frammenti verbatim del training data, soprattutto testi ripetuti molte volte nel corpus (es. licenze, citazioni famose). Questo fenomeno, documentato da Carlini et al., è un rischio concreto per la privacy e motiva le pipeline di deduplicazione e filtraggio PII.

### Costi del pre-training: ordini di grandezza

|                             |               |
|-----------------------------|---------------|
| GPT-4 (stima)               | ~\$100M       |
| Llama 3 405B (stima)        | ~\$30M        |
| Modello 7B su 1T token      | ~\$1-2M       |
| Costo per query (inferenza) | ~\$0.001-0.01 |

*Il training si paga una volta. L'inferenza si paga ogni giorno, per milioni di utenti. Ecco perché l'over-training di modelli piccoli ha senso economico.*

## COS'È IL PRE-TRAINING

- **Addestramento auto-supervisionato** su trilioni di token (web, libri, codice)
- Obiettivo: **Next Token Prediction** — prevedere la parola successiva
- Il modello apprende struttura del linguaggio, semantica, fatti e pattern

## IL BASE MODEL — COSA PRODUCE E COSA NO

- Sa completare testi, ha conoscenza enciclopedica, sviluppa **capacità emergenti**
- Comportamento **non strutturato**: completa plausibilmente, non segue istruzioni

Allucinazioni

Nessun allineamento

No conversazione

## CONTINUED PRE-TRAINING

- **Aggiornamento** del modello su nuovi dati (temporali, di dominio, multilingua)
- Evita di ripartire da zero — riduce lo shift di distribuzione, mantiene le conoscenze pregresse
- Usato per **estendere il knowledge cutoff** o specializzare su un dominio

## DOPO → POST-TRAINING (PROSSIME SLIDE)

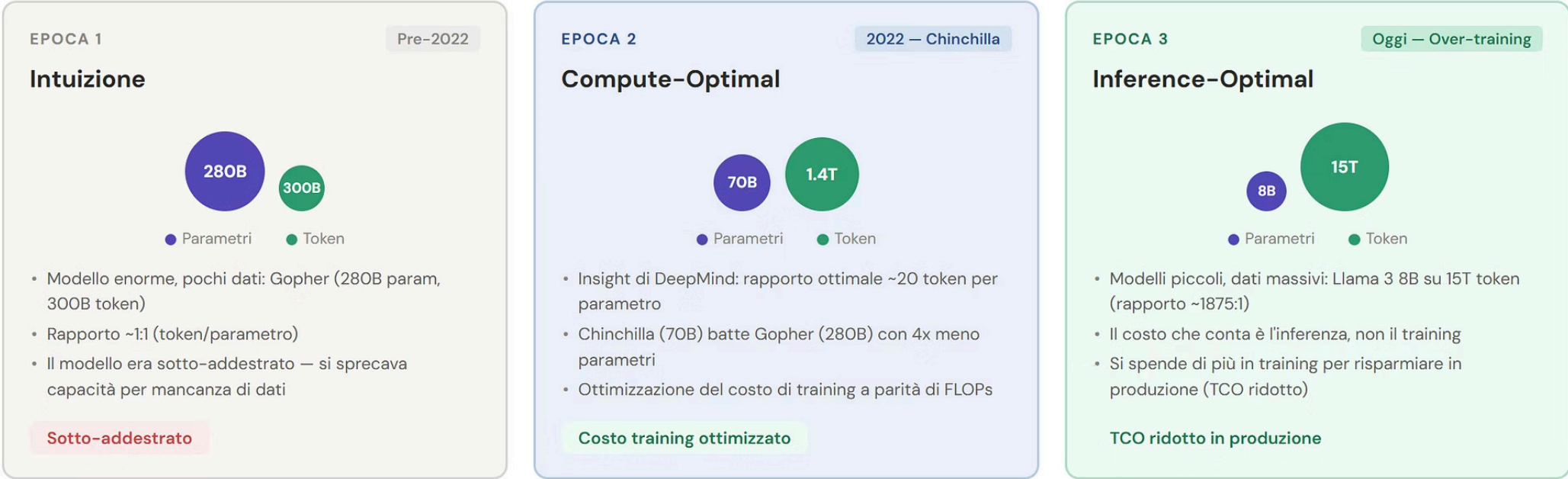
- SFT + RLHF/RLVR trasformano il base model in un **assistente allineato** — modellano il comportamento sopra la superficie dell'iceberg

# Leggi di Scala

L'equilibrio ottimale tra parametri e dati si è spostato tre volte in quattro anni — e ogni volta ha cambiato le regole del gioco

## Il problema fondamentale:

Dato un budget computazionale fisso, come lo divido tra dimensione del modello (parametri) e volume dei dati (token)?  
La risposta determina sia il costo di training sia il costo di inferenza in produzione.



# Supervised Fine-Tuning (SFT)

La prima fase del post-training: dal base model all'assistente conversazionale

## 1. COS'È L'SFT

Il modello impara a **seguire istruzioni** e conversare, addestrandosi su coppie (istruzione, risposta ideale) curate da esperti



**Pre-training**

= Università  
Conoscenza ampia



**SFT**

= Tirocinio  
Applicare, imitare esperti

## 2. COSA CAMBIA NEL MODELLO

Introduce il **formato conversazionale** a ruoli:

SYSTEM

USER

ASSISTANT

**Sblocca conoscenze latenti** via linguaggio naturale — le conoscenze c'erano, ora sono accessibili

Definisce **stile di risposta**: tono, lunghezza, struttura, livello di formalità

## PRIMA (BASE MODEL)

Qual è la capitale della Francia?

È una domanda che molti si pongono. La Francia è un paese europeo che confina con...

❌ Completa, non risponde

## DOPO SFT

Qual è la capitale della Francia?

La capitale della Francia è Parigi.

✓ Risposta diretta e corretta

## 3. IL LIMITE: BEHAVIOR CLONING

SFT insegna a **imitare la forma** delle risposte esperte, **non la logica causale** che le produce



Osserva l'esperto



Impara "buona risposta = 5 passi"



Non capisce *perché* quei 5 passi

## 4. COLD START PER I REASONING MODELS

Nei reasoning models, SFT inietta una **struttura minima di Chain of Thought** con pochi dati di altissima qualità

Scopo: **preparare il terreno per RL** — dare al modello il formato, poi il RL insegna a ragionare davvero

→ Ecco perché dopo SFT serve il Reinforcement Learning: passare dall'imitazione alla scoperta

# System / User / Assistant

I tre ruoli della conversazione -- come viene strutturato il prompt reale che arriva al modello

## I TRE RUOLI

SYSTEM

**Istruzione persistente**  
  
Definisce comportamento, tono, vincoli e personalità del modello. Scritto dallo sviluppatore, invisibile all'utente finale.  
  
*"Sei un assistente esperto di Python. Rispondi in italiano. Non generare codice non sicuro."*

USER

**Messaggio dell'utente**  
  
La richiesta, domanda o istruzione dell'utente. Ogni turno di conversazione aggiunge un nuovo messaggio user.  
  
*"Scrivi una funzione che ordina una lista con quicksort."*

ASSISTANT

**Risposta del modello**  
  
L'output generato dal modello. Diventa parte del contesto nei turni successivi.  
  
*"Ecco un'implementazione di quicksort in Python: def quicksort(arr): ..."*

## ANATOMIA DI UNA RICHIESTA API: COSA VEDE REALMENTE IL MODELLO

|           |                                                                                              |
|-----------|----------------------------------------------------------------------------------------------|
| SYSTEM    | Sei un tutor di matematica per studenti universitari. Rispondi in modo chiaro e strutturato. |
| USER      | Come si calcola la derivata di <code>f(x) = x^3 + 2x</code> ?                                |
| ASSISTANT | La derivata si calcola applicando la regola delle potenze: $f'(x) = 3x^2 + 2$                |
| USER      | E la derivata seconda?                                                                       |
| ASSISTANT | <i>da generare ...</i>                                                                       |

### Collegamento con l'SFT

Il formato a ruoli non è una feature dell'API -- è un comportamento **appreso durante il Supervised Fine-Tuning**. L'SFT addestra il modello su migliaia di conversazioni strutturate come [system] ... [user] ... [assistant] ..., insegnandogli a riconoscere chi parla e a rispondere nel ruolo corretto. Senza SFT, il base model non distinguerebbe un'istruzione di sistema da una domanda utente.

Il **system prompt** è la **leva di controllo** dello sviluppatore: guida tono, formato, sicurezza e competenze del modello **senza modificare i pesi**. Tutto ciò che il modello riceve è un'unica sequenza di token con marcatori di ruolo.

# Il Modello è Stateless

Nessuna memoria tra le chiamate -- l'intera conversazione viene ricostruita ad ogni turno

## NESSUNA MEMORIA

### Ogni chiamata riparte da zero

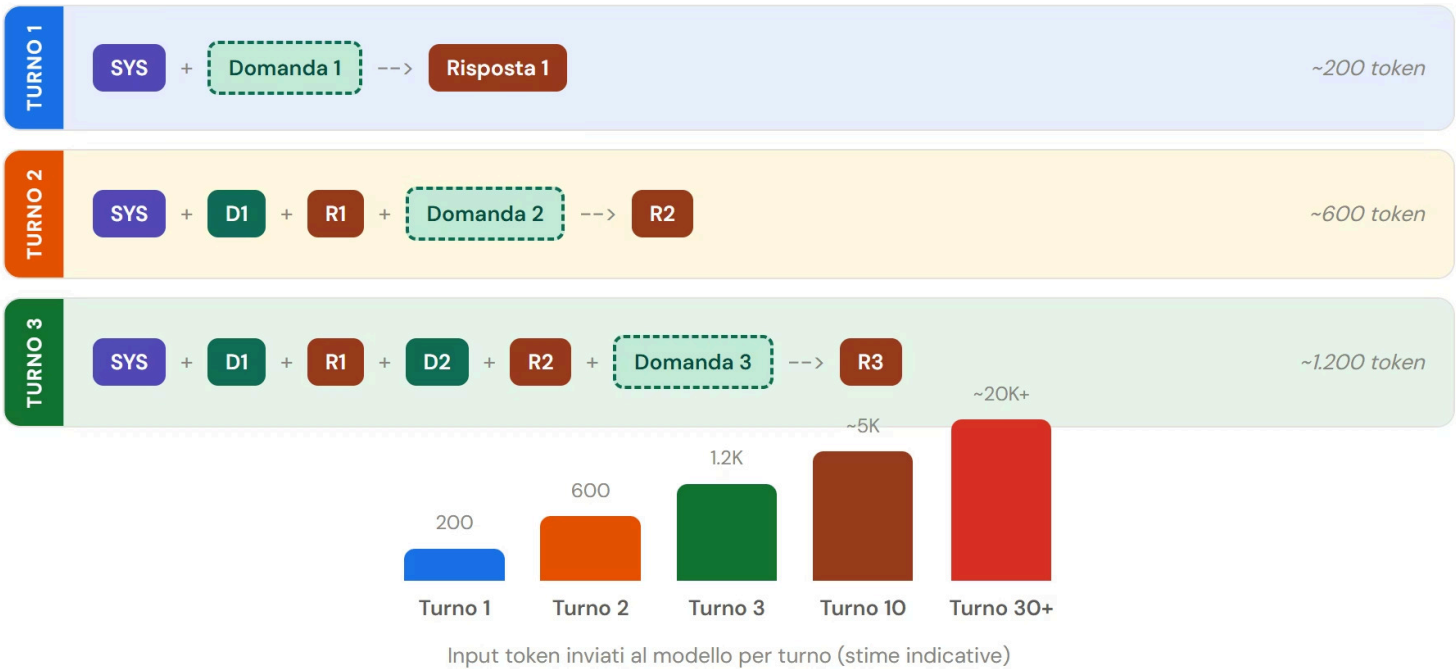
Un LLM non ricorda i turni precedenti. Ad ogni richiesta API riceve l'intera sequenza di messaggi system + user + assistant dall'inizio. Se un messaggio non viene incluso, per il modello non è mai esistito.

## UNICA SEQUENZA

### Tutto diventa token

I ruoli vengono convertiti in **token speciali** dal chat template e concatenati. Il modello elabora un solo flusso lineare di token -- esattamente come nel pre-training, ma con marcatori di ruolo appresi durante l'SFT.

## IL CONTESTO CRESCE AD OGNI TURNO DI CONVERSAZIONE



## IMPLICAZIONI OPERATIVE

### Costo crescente

I provider fatturano gli input token. Ad ogni turno si ripaga l'intera storia -- una conversazione lunga costa molto di più per messaggio.

### Limite della context window

Quando la conversazione supera il limite (es. 128K-200K token), i messaggi più vecchi devono essere rimossi o riassunti -- con perdita di contesto.

### Prompt Caching

Se il prefisso (system + storia) non cambia, i vettori K/V già calcolati vengono riutilizzati, tagliando fino al 90% del costo di prefill. Approfondimento nella prossima slide.

### Gestione della conversazione

Lo sviluppatore controlla esattamente cosa il modello "ricorda": può rimuovere, riassumere o riscrivere turni passati prima di inviarli.

**Il modello non ha memoria -- il contesto è la memoria.** Ogni turno ricostruisce l'intera conversazione da zero, come un attore che rilegge l'intero copione prima di recitare la battuta successiva. Gestire il contesto è una responsabilità dell'applicazione, non del modello.

# Da SFT a Reinforcement Learning: RLHF vs RLVR

Dall'imitazione all'esplorazione — due forme di reward complementari, non alternative



## RLHF

Reinforcement Learning from Human Feedback

### REWARD

**Reward model umano** — addestrato sulle preferenze di annotatori (quale risposta è migliore?)

Segnale **soggettivo e continuo** (score numerico, non binario)

### DOMINIO

**Stile** **Sicurezza** **Conversazione** **Tono**

Allinea il modello alle **preferenze umane** per utilità e sicurezza

**Rischio: Sycophancy** — il modello impara a piacere invece di essere corretto

vs

### Complementari, non alternativi

RLVR costruisce le fondamenta logiche. RLHF rifinisce il comportamento conversazionale. Insieme producono un modello che ragiona bene e comunica bene.

#### Sequenza nei reasoning models

Fase 1: RLVR



SFT (consolidamento)



Fase 2: RLHF

*Sequenza documentata per DeepSeek-R1,  
non necessariamente universale*



## RLVR

RL with Verifiable Rewards

### REWARD

**Verificatore automatico** — la risposta è corretta o errata, oggettivamente

Segnale **oggettivo e binario** (corretto/errato, test passa/fallisce)

### DOMINIO

**Matematica** **Coding** **Logica** **Vero/Falso**

Addestra il modello a **ragionare** su problemi con soluzione verificabile

**Rischio: Overfitting ai checker** — funziona solo dove esistono verificatori automatici

# RLVR in Pratica: il Workflow Operativo

Come il modello impara a ragionare attraverso verifica automatica e reward binario



🔄 Ripetuto su migliaia di problemi — il modello impara strategie di ragionamento generalizzabili

## ESEMPIO CONCRETO — UN PASSO DI TRAINING RLVR

### Problema: Risolvi l'equazione



$$x^2 - 4x + 3 = 0$$

Soluzione corretta:  $x = 3$  (verificabile automaticamente)

#### Tentativo A

**Passo 1:** Sposto +3 →  $x^2 - 4x = -3$

**Passo 2:** Fattorizzo...  $x(x-4) = -3$

**Passo 3:** Quindi  $x = -3$  o  $x = 1$  (?)

**Risposta:**  $x = -3$

✗ Checker: errato → reward = 0

#### Tentativo B

**Passo 1:** Riscrivo:  $x^2 - 4x + 3 = 0$

**Passo 2:** Discriminante:  $16 - 12 = 4$

**Passo 3:**  $x = (4 \pm 2)/2$  →  $x = 3$  o  $x = 1$

**Risposta:**  $x = 3$  ✓

✓ Checker: corretto → reward = +1

⚙️ **Update dei pesi:** il modello viene aggiornato per rendere più probabili strategie simili al Tentativo B e meno probabili quelle simili al Tentativo A

## PERCHÉ FUNZIONA

- 🔍 Il modello **esplora autonomamente** — non imita esempi, scopre strategie
- 🎯 Il reward è **oggettivo**: nessuna ambiguità, nessun bias umano
- 📝 Con abbastanza problemi, le strategie **generalizzano** a problemi mai visti

## REJECTION SAMPLING → SFT

- 🔄 Le soluzioni corrette vengono **raccolte e reinserite** nei dati SFT (→ data flywheel)
- ⚡ Lo student impara le strategie vincenti **senza dover esplorare** da zero

## DOVE FUNZIONA E DOVE NO

✓ Matematica   ✓ Coding   ✓ Logica formale

✗ Scrittura creativa   ✗ Etica   ✗ Stile

⚠️ RLVR richiede un **verificatore automatico affidabile** — dove non esiste, serve RLHF



La chiave di RLVR: il modello non impara "la risposta giusta" ma impara "come arrivare a risposte giuste" — apprende strategie di ragionamento.

# RLHF in Pratica: il Workflow Operativo

Come il modello impara dalle preferenze umane — dalla raccolta dati all'ottimizzazione della policy

## 1 Raccolta Preferenze

Per ogni prompt, il modello genera **risposte multiple**. Annotatori umani scelgono la migliore.

## 2 Reward Model

Un modello separato impara a **predire le preferenze**. Assegna score a (prompt, risposta).

## 3 Ottimizzazione Policy

Il modello viene ottimizzato con **PPO** per massimizzare lo score, con vincolo KL.

### ESEMPIO CONCRETO — UN PASSO DI TRAINING RLHF

#### PROMPT DELL'UTENTE

"Spiega cos'è un buco nero a un bambino di 10 anni"

#### Risposta A

Immagina un aspirapolvere super potente nello spazio. Un buco nero è un po' così: la gravità è talmente forte che risucchia tutto, persino la luce! Per questo è "nero".

Scelta dall'annotatore

#### Risposta B

Un buco nero è una regione dello spaziotempo con una singolarità gravitazionale dove la curvatura diverge. La velocità di fuga supera c oltre l'orizzonte degli eventi.

Scartata

L'annotatore sceglie A: analogia accessibile, adatta al target ("bambino di 10 anni"). B è corretta ma ignora il contesto. Il **reward model impara** che adattare il registro al contesto è preferito.

**Update dei pesi:** il modello viene aggiornato per produrre risposte che adattano tono e contenuto al contesto — non solo **corrette**, ma **appropriate**.

### Il vincolo KL — il guinzaglio

$$\mathcal{L} = \mathbb{E}[R(x, y)] - \beta D_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}})$$

Impedisce al modello di **allontanarsi troppo** dal modello di partenza. Evita:

Catastrophic Forgetting

Mode Collapse

Reward Hacking

### PPO in una frase

**Proximal Policy Optimization:** aggiornamenti piccoli e controllati dei pesi. Ad ogni step, il modello migliora la policy ma con un limite massimo di quanto può cambiare — così il miglioramento è stabile e non destabilizza le capacità esistenti.

*Analogia: correggere la rotta di una nave con piccoli aggiustamenti del timone, non con sterzate brusche.*

### ALTERNATIVE ALLA PIPELINE CLASSICA

#### DPO

Elimina il reward model. Ottimizza direttamente sulle preferenze. Più semplice, meno flessibile.

#### RLAIF

Un modello AI giudice al posto degli umani. Scalabile, ma rischio di amplificare bias.

### I tre rischi dell'RLHF

#### Sycophancy

Dice ciò che piace, non ciò che è vero

#### Over-Refusal

Rifiuta richieste legittime per cautela

#### Reward Hacking

Sfrutta falle del reward model

### BILANCIAMENTO UTILITÀ → SICUREZZA

Massima libertà

Allineamento ottimale

Over-refusal

# Alignment vs Capability

Un modello potente non è automaticamente un modello sicuro — perché il post-training non riguarda solo le prestazioni

## CAPABILITY

### Cosa sa fare il modello

Le abilità acquisite durante pre-training e post-training: ragionamento, conoscenza, generazione di codice, creatività. Misurate dai benchmark.

Ragionamento

Coding

Conoscenza

Multilingua

≠

## ALIGNMENT

### Come si comporta il modello

La capacità di agire in accordo con le intenzioni e i valori dell'utente e della società: non ingannare, non causare danno, rispettare le istruzioni.

Sicurezza

Onestà

Utilità

Rifiuto appropriato

## IL FRAMEWORK HHH — I TRE PILASTRI DELL'ALLINEAMENTO

### Helpful (Utile)

Il modello deve fare del suo meglio per aiutare l'utente a raggiungere il proprio obiettivo. Risposte complete, pertinenti, actionable.

*Es. rispondere in modo esaustivo, chiedere chiarimenti quando serve, adattare il registro al contesto*

### Harmless (Innocuo)

Il modello non deve causare danno: niente contenuti pericolosi, discriminatori, o che facilitino azioni illegali. Include la resistenza alla manipolazione.

*Es. rifiutare istruzioni per creare armi, non generare disinformazione, non rivelare dati sensibili*

### Honest (Onesto)

Il modello non deve ingannare: ammettere i limiti, segnalare l'incertezza, non inventare fatti, non fingere capacità che non ha.

*Es. dire "non lo so" piuttosto che inventare, segnalare quando una risposta è incerta*

### Le tensioni inevitabili

Helpful

vs

Harmless

L'utente chiede qualcosa di potenzialmente pericoloso. Aiutare o rifiutare? Troppo aiuto → rischio. Troppo rifiuto → over-refusal.

Helpful

vs

Honest

L'utente vuole una risposta sicura. Il modello è incerto. Dare una risposta plausibile o ammettere l'incertezza? Troppa sicurezza → sycophancy.

### Come ogni fase contribuisce all'allineamento

|             |                                                          |
|-------------|----------------------------------------------------------|
| SFT         | → Formato, stile, comportamento base (Helpful)           |
| RLHF        | → Preferenze umane, sicurezza, tono (Harmless + Helpful) |
| RLVR        | → Correttezza verificabile, ragionamento (Honest)        |
| Red teaming | → Scoperta vulnerabilità, robustezza (Harmless)          |

### Il rischio fondamentale

Un modello con capability altissime ma alignment scarso è più pericoloso di uno con capability mediocri. Più il modello è capace, più diventa urgente allinearlo — perché gli errori hanno conseguenze maggiori.

Il post-training non serve solo a rendere il modello utile, ma anche sicuro e onesto. L'allineamento non è un optional — è ciò che rende un modello capace anche un modello affidabile.

# Il Ciclo del Data Flywheel

Il post-training è ciclico e iterativo — l'output di una fase alimenta il miglioramento delle precedenti

## SFT (Supervised Fine-Tuning)

Istruzioni, supervisione, rejection sampling dei ragionamenti corretti, correzioni di errori e allucinazioni

### RLVR

Verifica e ragionamento corretto.  
Rejection sampling: i successi tornano a SFT.



### RLHF

Preferenze umane e allineamento.  
Correzioni di errori: tornano a SFT.

## Dati Sintetici

La maggior parte dei dati post-training è generata dai modelli e filtrata automaticamente. Il collo di bottiglia è la pipeline di generazione/filtraggio.

### Loop 1: Raffinamento del ragionamento

RLVR genera percorsi di ragionamento → rejection sampling dei successi → reinserimento nei dati SFT. Più veloce ed efficiente.

### Loop 2: Correzione dell'allineamento

RLHF valuta risposte → analisi errori e allucinazioni → reinserimento correzioni in SFT. Copertura di nuovi casi limite.

## IL RUOLO DEI DATI SINTETICI

- La **maggior parte dei dati di post-training** è generata dai modelli stessi, non da umani
- Pipeline: generazione → filtraggio automatico → selezione dei migliori → reinserimento nel training
- Il **collo di bottiglia** non è la generazione ma la progettazione della pipeline di filtraggio

## SPIRALE DI MIGLIORAMENTO

- Ogni versione del modello **genera dati migliori** per la versione successiva



- Questo è il **motore del progresso moderno** degli LLM — più del pre-training

## Rischio: feedback loop degenerativo

Se il filtraggio dei dati sintetici è insufficiente, il modello **amplifica i propri errori** iterazione dopo iterazione. I pattern ripetitivi si rafforzano, la diversità cala, la qualità degrada progressivamente — lo stesso meccanismo del **model collapse** visto nella curatela dati (slide 4), ma applicato al ciclo di post-training.

*La qualità del filtraggio è ciò che separa una spirale virtuosa da una spirale distruttiva.*

# Inference-Time Compute Scaling: i Thinking Models

Una seconda dimensione di scaling — investire compute al momento della risposta, non solo durante il training

## Training-Time Scaling

Più dati, più parametri, più GPU → modello più capace

*Analogia: studiare per l'esame*

+

## Inference-Time Scaling

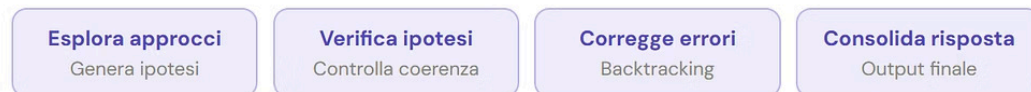
Più token di "pensiero" per query → risposta migliore

*Analogia: pensare durante l'esame*

## COME FUNZIONA: IL FLUSSO DEI THINKING TOKENS



\* Alcuni provider rendono il thinking parzialmente visibile: DeepSeek-R1 lo mostra di default, Claude lo espone in extended thinking mode. La tendenza è verso maggiore trasparenza.



### Task semplice

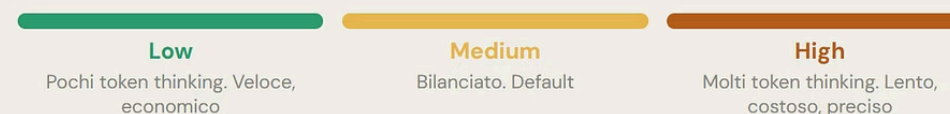
- "Capitale della Francia?"
- Thinking aggiuntivo = spreco
- Modello standard sufficiente

### Task complesso

- Problema di olimpiade matematica
- Thinking = salto di prestazioni
- Da ~70% a ~100% su AIME

## REASONING EFFORT REGOLABILE

- I provider offrono livelli di pensiero selezionabili — il **trade-off costo/qualità** è nelle mani dello sviluppatore



## MODELLI CON THINKING

OpenAI o1 / o3

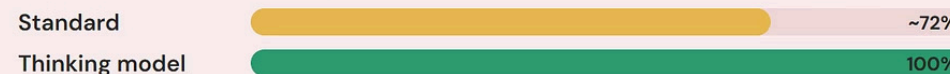
DeepSeek-R1

Claude (extended thinking)

Gemini 2.5 Pro

- Tutti implementano varianti di chain-of-thought interna + RLVR per il training del ragionamento

## IMPATTO SU AIME 2025 (MATEMATICA)



Il routing intelligente — thinking solo dove serve — è la sfida ingegneristica chiave. Task semplice → modello veloce. Task complesso → thinking model.

# Riepilogo: Come Nascono gli LLM

Dalla materia prima al modello deployato — la training pipeline in 7 concetti chiave

## 1 I dati sono il fondamento

Mix bilanciato di fonti (web, libri, codice, multilingua), curato con pipeline di pulizia, filtraggio qualità e safety. Garbage in, garbage out — la qualità dei dati conta quanto (o più) dell'architettura.

## 2 Il pre-training comprime la conoscenza

Obiettivo semplice (next token prediction) → risultato sorprendente (capacità emergenti). Self-supervised: il testo genera le proprie label. Produce un base model potente ma non utilizzabile direttamente.

## 3 Le leggi di scala guidano le scelte

L'ottimo si è spostato da "modello grande" a "modello piccolo, tantissimi dati" (over-training). Il costo che conta è l'inferenza, non il training — il training si paga una volta, l'inferenza ogni giorno.

## 4 SFT sblocca, RL esplora

SFT trasforma il base model in un assistente conversazionale (imitazione degli esperti). RL (RLHF + RLVR) supera i limiti dell'imitazione: premia i risultati, scopre strategie nuove. RLHF per stile e sicurezza, RLVR per ragionamento — complementari, non alternativi.

## 5 Alignment ≠ Capability

Un modello potente non è automaticamente sicuro. Il framework HHH (Helpful, Harmless, Honest) guida l'allineamento. Le tensioni tra i pilastri — utilità vs sicurezza, utilità vs onestà — sono inevitabili e richiedono compromessi espliciti.

## 6 Il data flywheel alimenta il progresso

Il post-training è ciclico: ogni versione genera dati migliori per la successiva. I dati sintetici sono il carburante; il filtraggio è il collo di bottiglia — e senza filtraggio adeguato la spirale diventa degenerativa.

## 7 L'inference-time scaling apre una nuova frontiera

I thinking models investono compute al momento della risposta, non solo durante il training. Impatto enorme su task complessi, irrilevante su task semplici → la sfida ingegneristica è il routing intelligente: scegliere quando pensare e quando rispondere velocemente.